

**VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY**



HUYNH HAI QUAN – 525H0088

MIDTERM ESSAY

APPLIED LINEAR ALGEBRA FOR IT

**HO CHI MINH CITY, 2026
VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY**



HUYNH HAI QUAN – 525H0088

MIDTERM ESSAY

APPLIED LINEAR ALGEBRA FOR IT

Advised by

Professor Pham Quoc Duy

HO CHI MINH CITY, 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Mr. Pham Quoc Duy, my instructor and mentor, for his valuable guidance and support throughout the midterm report for the Applied Linear Algebra course. He has been incredibly helpful and patient in providing me with constructive feedback and suggestions to improve my work. He has also encouraged me to deepen my understanding of mathematical concepts and how to effectively apply them using Python. I have learned a great deal from his expertise in this field, and I am honored and privileged to have him as my teacher.

Ho Chi Minh city, 5th April 2026.

Author

(Signature and full name)

Quan

Huynh Hai Quan

DECLARATION OF AUTHORSHIP

I hereby declare that this is my own project, guided by Mr. Pham Quoc Duy. The programming logic, source code, and results contained herein are my original work and have not been published or submitted in any form before.

Any mathematical concepts, programming libraries, or references utilized in this essay are standard course materials or appropriately applied tools for this specific assignment.

I take full responsibility for the content and academic integrity of my project. Ton Duc Thang University is not liable for any infringing rights or copyright issues that may arise during the implementation process.

Ho Chi Minh city, 3rd September 2023

Author

(Signature and full name)

Quan

Huynh Hai Quan

APPLIED LINEAR ALGEBRA FOR IT

ABSTRACT

This project focuses on the practical application of Linear Algebra concepts using the Python programming language and the NumPy library. The primary objective is to demonstrate proficiency in matrix operations, including addition, transpose, and multiplication, while resolving mathematical constraints such as dimension mismatches. Additionally, the project implements algorithmic logic to extract specific data types—such as prime numbers and odd sequences—from generated matrices. The results are documented through a series of five distinct execution trials to verify the consistency and accuracy of the implemented methodology.

TABLE OF CONTENT

LIST OF FIGURES.....	v
ABBREVIATIONS.....	vi
CHAPTER 1. METHODOLOGY OF SOLVING TASKS	1
1.1 Task 1a: Matrix Addition and Transpose Operations	1
1.2 Task 1b: Submatrix Selection and Multiplication.....	1
1.3 Task 1c: Extraction and Sorting of Odd Integers.....	2
1.4 Task 1d: Vector Extraction and Prime Number Summation	3
1.5 Task 1e: Row Analysis for Prime Number Density	4
1.6 Task 1f: Contiguous Odd Number Sequence Identification.....	5
CHAPTER 2. RESULTS.....	6
2.1 Trial 1 Results	6
2.2 Trial 2 Results	7
2.3 Trial 3 Results	8
2.4 Trial 4 Results	9
2.5 Trial 5 Results	10
REFERENCES.....	11

LIST OF FIGURES

Figure 2.1: Terminal output for Trial 1: Matrix initialization and Tasks 1a-1f	6
Figure 2.2: Terminal output for Trial 2: Matrix initialization and Tasks 1a-1f	7
Figure 2.3: Terminal output for Trial 3: Matrix initialization and Tasks 1a-1f	8
Figure 2.4: Terminal output for Trial 4: Matrix initialization and Tasks 1a-1f	9
Figure 2.5: Terminal output for Trial 5: Matrix initialization and Tasks 1a-1f	10

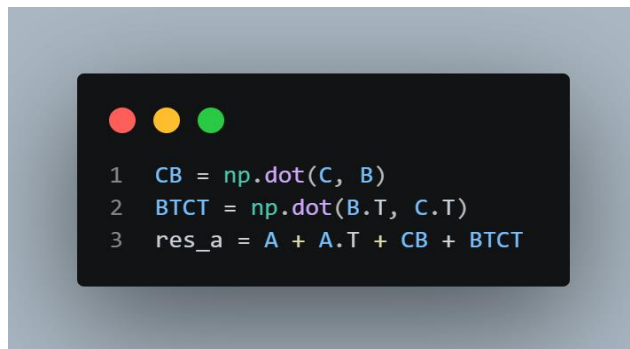
ABBREVIATIONS

IT	Information Technology
NumPy	Numerical Python
1D / 2D	One – Dimensionals / Two – Dimensionals

CHAPTER 1. METHODOLOGY OF SOLVING TASKS

1.1 Task 1a: Matrix Addition and Transpose Operations

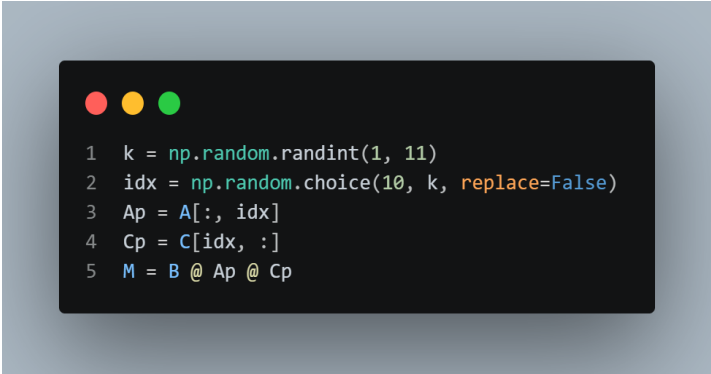
Objective: Perform a composite matrix calculation involving addition, transpositions, and dot products to produce a final 10×10 result.



- **CB:** Uses `np.dot(C, B)` to calculate the product of matrix C (10×2) and B (2×10), which results in a 10×10 matrix.
- **BTCT:** Uses `np.dot(B.T, C.T)` to multiply the transpose of B by the transpose of C .
- **AT:** Computes the transpose of the (10×10) matrix A , swapping its rows and columns.
- **res_a:** Combines all terms (A , A^T , CB , and $B^T C^T$) through matrix addition to generate the final output.

1.2 Task 1b: Submatrix Selection and Multiplication

Objective: Randomly select k matching columns and rows from A and C to compute the product matrix $M = B.A'.C'$



```

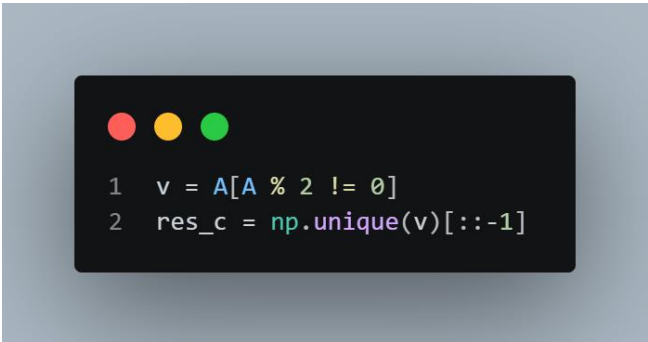
1 k = np.random.randint(1, 11)
2 idx = np.random.choice(10, k, replace=False)
3 Ap = A[:, idx]
4 Cp = C[idx, :]
5 M = B @ Ap @ Cp

```

- **k:** Generates a random integer between 1 and 10 to define the size of the submatrices.
- **idx:** Uses `np.random.choice` to pick k unique, non-repeating indices from the range of 10.
- **Ap & Cp:** These lines use NumPy slicing to create submatrix A' (all rows, selected columns) and C' (selected rows, all columns).
- **M:** Uses the `@` operator to perform sequential matrix multiplication ($B \times A' \times C'$), resulting in a 2×2 matrix.

1.3 Task 1c: Extraction and Sorting of Odd Integers

Objective: Isolate all odd integers within matrix **A**, ensuring the resulting vector **v** contains only unique values sorted in descending order.



```

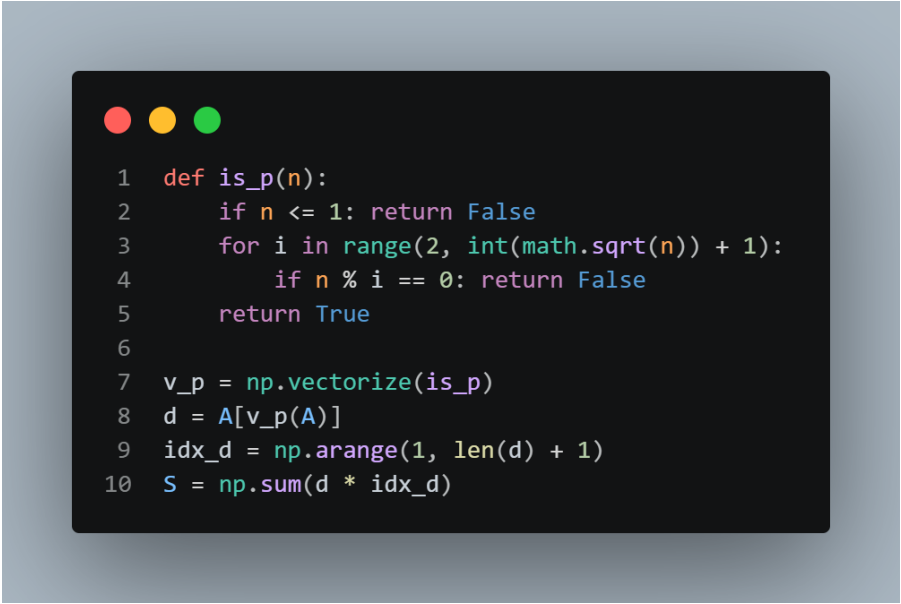
1 v = A[A % 2 != 0]
2 res_c = np.unique(v)[::-1]

```

- **v:** Uses the modulo operator `%` within a boolean mask to extract all elements in **A** that are not divisible by 2.
- **res_c:** Wraps the vector in `np.unique()` to discard duplicate values and then applies the `[::-1]` slice to reverse the default ascending sort into a descending one.

1.4 Task 1d: Vector Extraction and Prime Number Summation

Objective: Extract all prime numbers from **A** into vector ***d*** and compute a weighted sum **S** based on the position of each element.



```

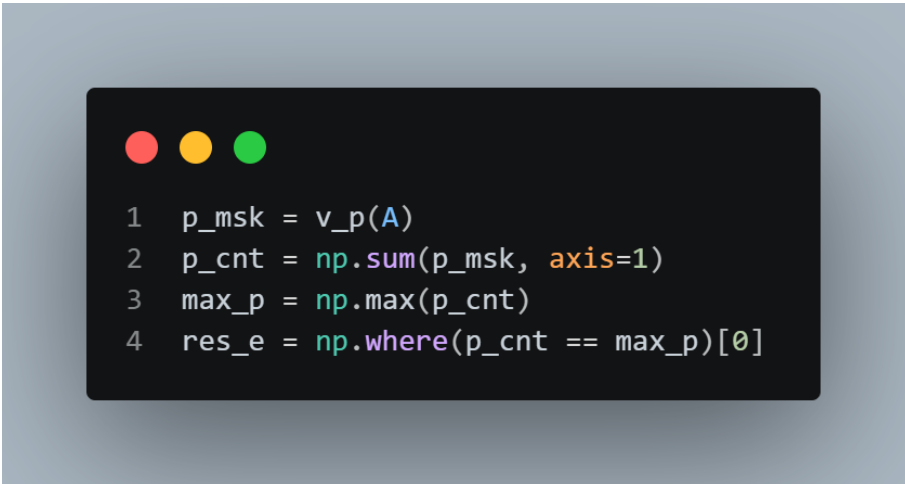
1  def is_p(n):
2      if n <= 1: return False
3      for i in range(2, int(math.sqrt(n)) + 1):
4          if n % i == 0: return False
5      return True
6
7  v_p = np.vectorize(is_p)
8  d = A[v_p(A)]
9  idx_d = np.arange(1, len(d) + 1)
10 S = np.sum(d * idx_d)

```

- **is_p(n):** A custom function that identifies prime numbers by checking for divisors up to the square root of n .
- **v_p:** Converts the scalar `is_p` function into a vectorized version that can be applied across the entire NumPy array.
- **d:** Filters matrix **A** through the `v_p` mask to store every identified prime number in a 1D array.
- **idx_d:** Generates a sequence of 1-based indices (1, 2, 3...) matching the length of the prime vector **d**.
- **S:** Multiplies each prime d_i by its index i and sums the results using `np.sum()`.

1.5 Task 1e: Row Analysis for Prime Number Density

Objective: Identify the specific rows in matrix **A** that contain the highest frequency of prime numbers.

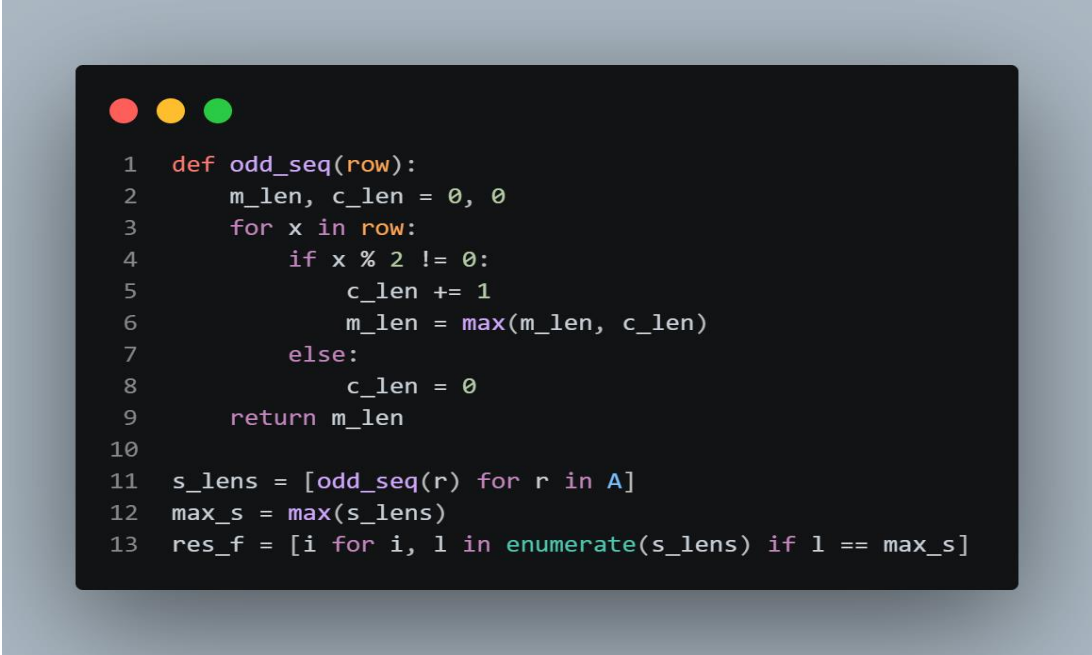


```
1 p_msk = v_p(A)
2 p_cnt = np.sum(p_msk, axis=1)
3 max_p = np.max(p_cnt)
4 res_e = np.where(p_cnt == max_p)[0]
```

- **p_msk:** Creates a boolean matrix where each entry is True if the corresponding value in **A** is prime.
- **p_cnt:** Sums the boolean values along each row (**axis=1**) to get a count of primes per row.
- **max_p:** Calculates the maximum value present in the **p_cnt** array.
- **res_e:** Uses **np.where** to find all row indices where the prime count equals **max_p**.

1.6 Task 1f: Contiguous Odd Number Sequence Identification

Objective: Scan matrix **A** to find rows containing the longest sequence of consecutive odd integers.



```
1  def odd_seq(row):
2      m_len, c_len = 0, 0
3      for x in row:
4          if x % 2 != 0:
5              c_len += 1
6              m_len = max(m_len, c_len)
7          else:
8              c_len = 0
9      return m_len
10
11 s_lens = [odd_seq(r) for r in A]
12 max_s = max(s_lens)
13 res_f = [i for i, l in enumerate(s_lens) if l == max_s]
```

- **odd_seq(row):** Iterates through a row to calculate the length of consecutive odd numbers, resetting the count whenever an even number is encountered.
- **s_lens:** A list comprehension that applies the sequence counting logic to every row in matrix **A**.
- **max_s:** Finds the highest sequence length achieved across the entire matrix.
- **res_f:** Identifies and stores the indices of all rows that match the **max_s** length.

CHAPTER 2. RESULTS

2.1 Trial 1 Results

```

Input Matrix A:
[[ 1 49 24 96 18 88 75 38 88 22]
 [ 5 96  7 43 65 60 56  7 60 79]
 [21 41 13  4 62 92 55 95 10  1]
 [53 24 41  7 72 33 11 80 34 77]
 [68 39 86  8 29 90 67 46 51 93]
 [44 52 37 59 33 26 89 84 59 65]
 [52 29 76 19 99 40 67 42 57 93]
 [48 53 56 28  9 61 13 52 62 10]
 [24 21 30 43 34 71 53 50 93 56]
 [20 19 22 96 27 96 80 62 51  6]]

Input Matrix B:
[[ 3 10 25 20 13 16  6 13 14 18]
 [ 3 30  2 26 19  3 23  1 27  9]]

Input Matrix C:
[[ 4  4]
 [18  9]
 [16  2]
 [19 17]
 [18 19]
 [17  4]
 [11  9]
 [13  8]
 [ 8  8]
 [10  1]]

```

```

Task 1a Result:
[[ 50 295 207 441 325 271 303 205 324 183]
 [295 1092 736 1361 1259 717 780 673 896 633]
 [207 736 834 926 882 824 566 702 534 581]
 [441 1361 926 1658 1504 891 989 840 1170 894]
 [325 1259 882 1504 1248 765 1025 629 1106 764]
 [271 717 824 891 765 620 526 602 628 666]
 [303 780 566 989 1025 526 680 469 739 535]
 [205 673 702 840 629 602 469 458 622 509]
 [324 896 534 1170 1106 628 739 622 842 490]
 [183 633 581 894 764 666 535 509 490 390]]

Task 1b Result with k=10 and indices=[8 0 7 3 6 5 4 1 9 2]:
[[870996 514156]
 [943037 552231]]

Task 1c Result for Vector v:
[99 95 93 89 79 77 75 71 67 65 61 59 57 55 53 51 49 43 41 39 37 33 29 27
 21 19 13 11  9  7  5  1]

Task 1d Result for S with 27 primes found: 16616

Task 1e Result for rows with max prime count of 5:
Row 2: [ 5 96  7 43 65 60 56  7 60 79]

Task 1f Result for rows with longest odd sequence of 3:
Row 2: [ 5 96  7 43 65 60 56  7 60 79]
Row 3: [21 41 13  4 62 92 55 95 10  1]
Row 6: [44 52 37 59 33 26 89 84 59 65]
Row 8: [48 53 56 28  9 61 13 52 62 10]

```

Figure 2.1 Terminal output for Trial 1: Matrix initialization and Tasks 1a-1f

2.2 Trial 2 Results

```

Input Matrix A:
[[ 3 86 2 43 46 10 42 89 25 76]
 [49 21 100 11 94 33 50 47 45 77]
 [31 37 96 60 25 45 73 94 92 89]
 [45 28 52 77 51 24 5 70 31 23]
 [44 34 53 37 69 21 27 49 16 20]
 [ 6 4 87 90 31 41 15 23 19 1]
 [58 12 65 39 3 93 32 16 29 41]
 [10 63 82 1 36 84 46 14 66 82]
 [28 48 76 19 8 49 41 55 1 5]
 [89 34 72 44 35 36 69 9 22 73]]

Input Matrix B:
[[19 15 17 1 21 10 16 4 9 12]
 [28 11 27 22 18 28 30 5 25 30]]

Input Matrix C:
[[ 4 15]
 [ 9 8]
 [ 8 15]
 [ 4 12]
 [ 1 6]
 [ 6 12]
 [16 9]
 [ 8 18]
 [16 12]
 [10 13]]

```

```

Task 1a Result:
[[ 998 755 1078 834 631 926 1170 846 1104 1217]
 [ 755 488 791 416 542 573 785 504 746 752]
 [1078 791 1274 842 695 1058 1231 905 1211 1228]
 [ 834 416 842 690 521 760 682 551 666 771]
 [ 631 542 695 521 396 572 724 611 735 691]
 [ 926 573 1058 760 572 874 976 775 918 933]
 [1170 785 1231 682 724 976 1116 839 1055 1122]
 [ 846 504 905 551 611 775 839 272 767 832]
 [1104 746 1211 666 735 918 1055 767 890 994]
 [1217 752 1228 771 691 933 1122 832 994 1166]]

Task 1b Result with k=8 and indices=[7 5 1 2 4 8 9 3]:
[[345387 559103]
 [597754 984959]]

Task 1c Result for Vector v:
[93 89 87 77 73 69 65 63 55 53 51 49 47 45 43 41 39 37 35 33 31 29 27 25
 23 21 19 15 11 9 5 3 1]

Task 1d Result for S with 27 primes found: 14282

Task 1e Result for rows with max prime count of 4:
Row 1: [ 3 86 2 43 46 10 42 89 25 76]
Row 3: [31 37 96 60 25 45 73 94 92 89]
Row 6: [ 6 4 87 90 31 41 15 23 19 1]

Task 1f Result for rows with longest odd sequence of 6:
Row 5: [44 34 53 37 69 21 27 49 16 20]
Row 6: [ 6 4 87 90 31 41 15 23 19 1]

```

Figure 2.2 Terminal output for Trial 2: Matrix initialization and Tasks 1a-1f

2.3 Trial 3 Results

```

Input Matrix A:
[[ 51  20  28  88  72  35  92  68  46  44]
 [  1  80  84  82  35  67  27  9  33  22]
 [ 97  66  87  45  94  17  29  74  34  69]
 [ 45  95  38  47 100  90  36  99  14  73]
 [ 58  90  12  50  84  88  78  68  89  44]
 [ 79  27  33  1  58  10  99  9  40  64]
 [ 28  36  11  41  40  26  77  46  73  44]
 [  5  13  91  22  22  57  45  94  53  68]
 [ 30  84  83  95  61  81  44  39  44  95]
 [ 46  8  14  49  32  48  80  53  45  32]]

Input Matrix B:
[[20  9 17 27 24 26 22  1  3 27]
 [28 23  4  4 19 15 10 22 24  3]]

Input Matrix C:
[[16  3]
 [19 14]
 [17 16]
 [ 7  9]
 [ 4  3]
 [14 15]
 [ 2 14]
 [ 7  8]
 [ 5  5]
 [16 19]]

```

```

Task 1a Result:
[[ 910 1006 1197  969  735 1275  934  519  436 1383]
 [1006 1146 1050 1016  952 1269  961  596  670 1166]
 [1197 1050  880  761  898 1030  664  685  657  938]
 [ 969 1016  761  544  609  846  431  547  501  846]
 [ 735  952  898  609  474  916  550  480  449  938]
 [1275 1269 1030  846  916 1198  845  712  728 1236]
 [ 934  961  664  431  550  845  522  635  619  762]
 [ 519  596  685  547  480  712  635  554  420  768]
 [ 436  670  657  501  449  728  619  420  358  794]
 [1383 1166  938  846  938 1236  762  768  794 1042]]

Task 1b Result with k=2 and indices=[7 5]:
[[189112 207566]
 [172193 188419]]

Task 1c Result for Vector v:
[99 97 95 91 89 87 83 81 79 77 73 69 67 61 57 53 51 49 47 45 41 39 35 33
 29 27 17 13 11  9  5  1]

Task 1d Result for S with 17 primes found: 7761

Task 1e Result for rows with max prime count of 3:
Row 3: [97 66 87 45 94 17 29 74 34 69]
Row 7: [28 36 11 41 40 26 77 46 73 44]
Row 8: [ 5 13 91 22 22 57 45 94 53 68]

Task 1f Result for rows with longest odd sequence of 5:
Row 2: [ 1 80 84 82 35 67 27  9 33 22]

```

Figure 2.3 Terminal output for Trial 3: Matrix initialization and Tasks 1a-1f

2.4 Trial 4 Results

```

Input Matrix A:
[[ 97  2 28 68 24 65 54 27 63 12]
 [ 31 39 55 27 70 18 66 16 79 7]
 [ 63 74 97 7 38 61 60 63 46 19]
 [ 37 57 40 41 43 11 93 41 3 2]
 [ 73 46 22 5 44 28 48 93 35 68]
 [ 39 98 53 94 22 2 99 13 79 64]
 [ 1 28 31 64 78 89 100 44 82 69]
 [ 4 74 63 82 81 39 5 95 35 98]
 [ 49 16 29 48 53 73 15 93 61 43]
 [ 50 61 6 49 83 28 59 79 59 78]]

Input Matrix B:
[[26 28 30 29 29 13 14 8 28 24]
 [21 19 7 9 9 13 13 4 10 23]]

Input Matrix C:
[[11 8]
 [11 17]
 [13 12]
 [13 5]
 [2 2]
 [11 18]
 [5 11]
 [17 5]
 [7 10]
 [11 15]]

```

```

Task 1a Result:
[[1102 1136 1067 939 582 1015 674 698 892 1111]
 [1136 1340 1170 1015 682 1130 818 817 959 1316]
 [1067 1170 1142 957 619 895 656 823 839 1048]
 [ 939 1015 957 926 546 820 648 785 758 932]
 [ 582 682 619 546 240 583 424 736 457 699]
 [1015 1130 895 820 583 758 784 498 861 1108]
 [ 674 818 656 648 424 784 626 436 575 850]
 [ 698 817 823 785 736 498 436 502 750 848]
 [ 892 959 839 758 457 861 575 750 714 958]
 [1111 1316 1048 932 699 1108 850 848 958 1374]]

Task 1b Result with k=5 and indices=[0 2 6 1 9]:
[[532489 668559]
 [293262 375389]]

Task 1c Result for Vector v:
[99 97 95 93 89 83 81 79 73 69 65 63 61 59 57 55 53 49 43 41 39 37 35 31
 29 27 19 15 13 11 7 5 3 1]

Task 1d Result for S with 35 primes found: 29862

Task 1e Result for rows with max prime count of 7:
Row 4: [37 57 40 41 43 11 93 41 3 2]

Task 1f Result for rows with longest odd sequence of 6:
Row 4: [37 57 40 41 43 11 93 41 3 2]
Row 9: [49 16 29 48 53 73 15 93 61 43]

```

Figure 2.4 Terminal output for Trial 4: Matrix initialization and Tasks 1a-1f

2.5 Trial 5 Results

```

Input Matrix A:
[[81 99 66 89 36 67 55 82 33 47]
 [ 6  5 49 48 38 55 52 78 51 78]
[26 67 82 15 82 75 19 69 26 17]
 [ 2 88 79 44 87 37 68 22 28 53]
[89 74 44 62 27 85 86 68 52 30]
[85 94 90 86 29 78 49 78 79 67]
[57 23 82 98 37 35 35 94 70 15]
[41 15 64 43  2 59 30 23 14 63]
 [ 5  6 41 33 26 70 30 35 89 76]
[35 63 58 74 68  4 98 20  2 94]]

Input Matrix B:
[[ 2  9 13  4  1 27  1  3 30 23]
[19 20 19 29 27 20 10  2 28 17]]

Input Matrix C:
[[ 5 16]
[17 19]
[11 15]
 [ 4 17]
 [ 6  2]
 [ 2 14]
 [ 4  1]
 [ 4 17]
[13  7]
 [ 3 18]]

```

```

Task 1a Result:
[[ 790  865  768  906  612  877  304  501  795  817]
 [ 865 1076 1097 1131  736 1286  338  558 1356 1242]
 [ 768 1097 1020  948  658 1054  333  571 1119  964]
 [ 906 1131  948 1106  694  985  385  620  912 1042]
 [ 612  736  658  694  174  696  180  555  516  759]
 [ 877 1286 1054  985  696  824  354  619 1092  796]
 [ 304  338  333  385  180  354  98  312  331  405]
 [ 501  558  571  620  555  619  312  138  698  509]
 [ 795 1356 1119  912  516 1092  331  698 1350 1090]
 [ 817 1242  964 1042  759  796  405  509 1090  938]]

Task 1b Result with k=3 and indices=[0 7 1]:
[[141372 271887]
[271692 518766]]

Task 1c Result for Vector v:
[99 89 87 85 81 79 75 69 67 63 59 57 55 53 51 49 47 43 41 37 35 33 29 27
 23 19 17 15  5]

Task 1d Result for S with 26 primes found: 13901

Task 1e Result for rows with max prime count of 5:
Row 8: [41 15 64 43  2 59 30 23 14 63]

Task 1f Result for rows with longest odd sequence of 3:
Row 3: [26 67 82 15 82 75 19 69 26 17]
Row 7: [57 23 82 98 37 35 35 94 70 15]

```

Figure 2.5 Terminal output for Trial 5: Matrix initialization and Tasks 1a-1f

REFERENCES

- [1] Pham Quoc Duy. (2026). *Applied Linear Algebra for IT: Midterm Essay Instructions*. Ton Duc Thang University, Faculty of Information Technology.
- [2] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). *Array programming with NumPy*. *Nature*, 585(7825), 357-362.
- [3] NumPy Documentation. (2026). *NumPy v2.0 Manual*. Retrieved from <https://www.google.com/search?q=https://numpy.org/doc/>
- [4] Python Software Foundation. (2026). *The Python Standard Library: Math Module*. Retrieved from <https://www.google.com/search?q=https://docs.python.org/3/library/math.html>